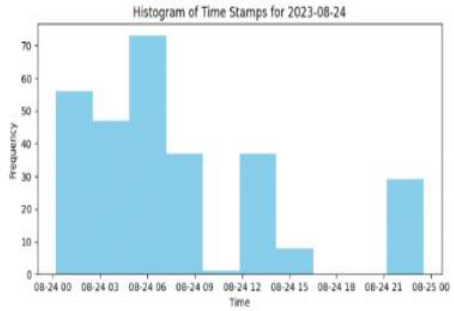
Prompt Data Analysis

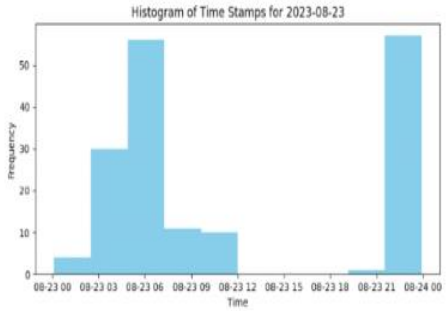
By: Jefferson Le

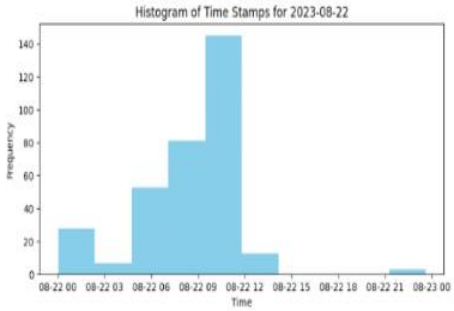
Preprocessing Data

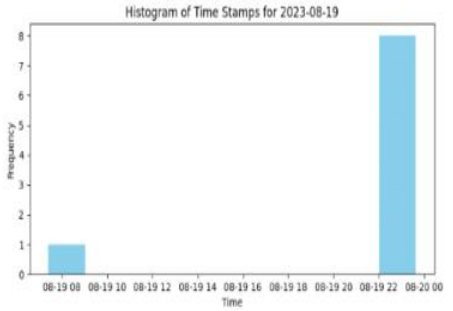
- I located all the features that contained NaN or missing values
- Initially removed all missing values, and noticed that there it reduced the amount of data by about 50%
- Realized that most of the missing data came from the stdout and stderr features.
- I rationalized that since the stdout feature had a lot of missing values, it was because the code produced by gpt did not compile, so therefore there wouldn't be a stdout.
- Chose to keep the missing value from just the stdout feature.
- Converted timestamp to datetime using pd.to_datetime

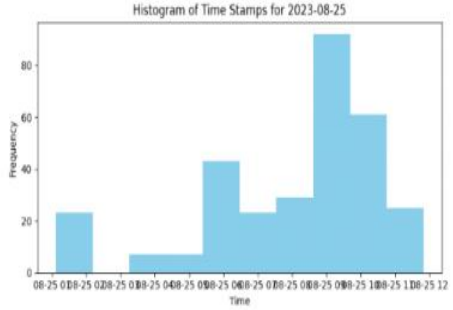
Data Visualization

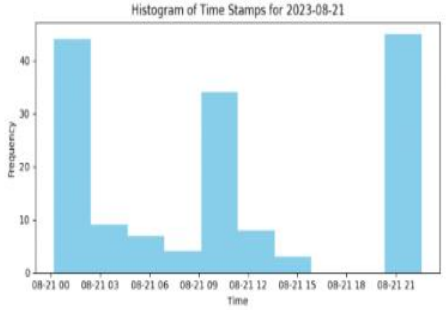


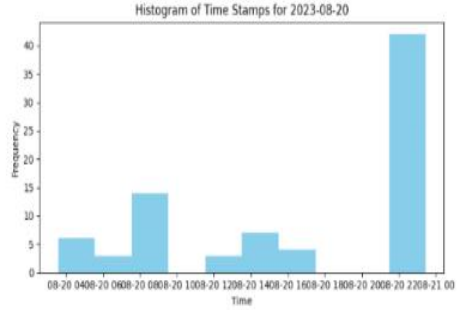


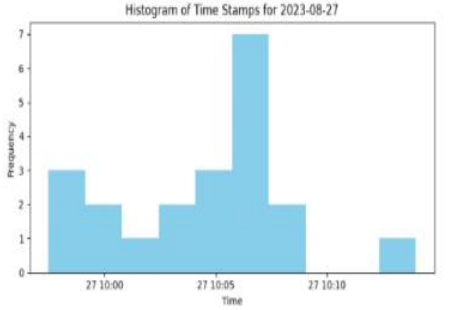




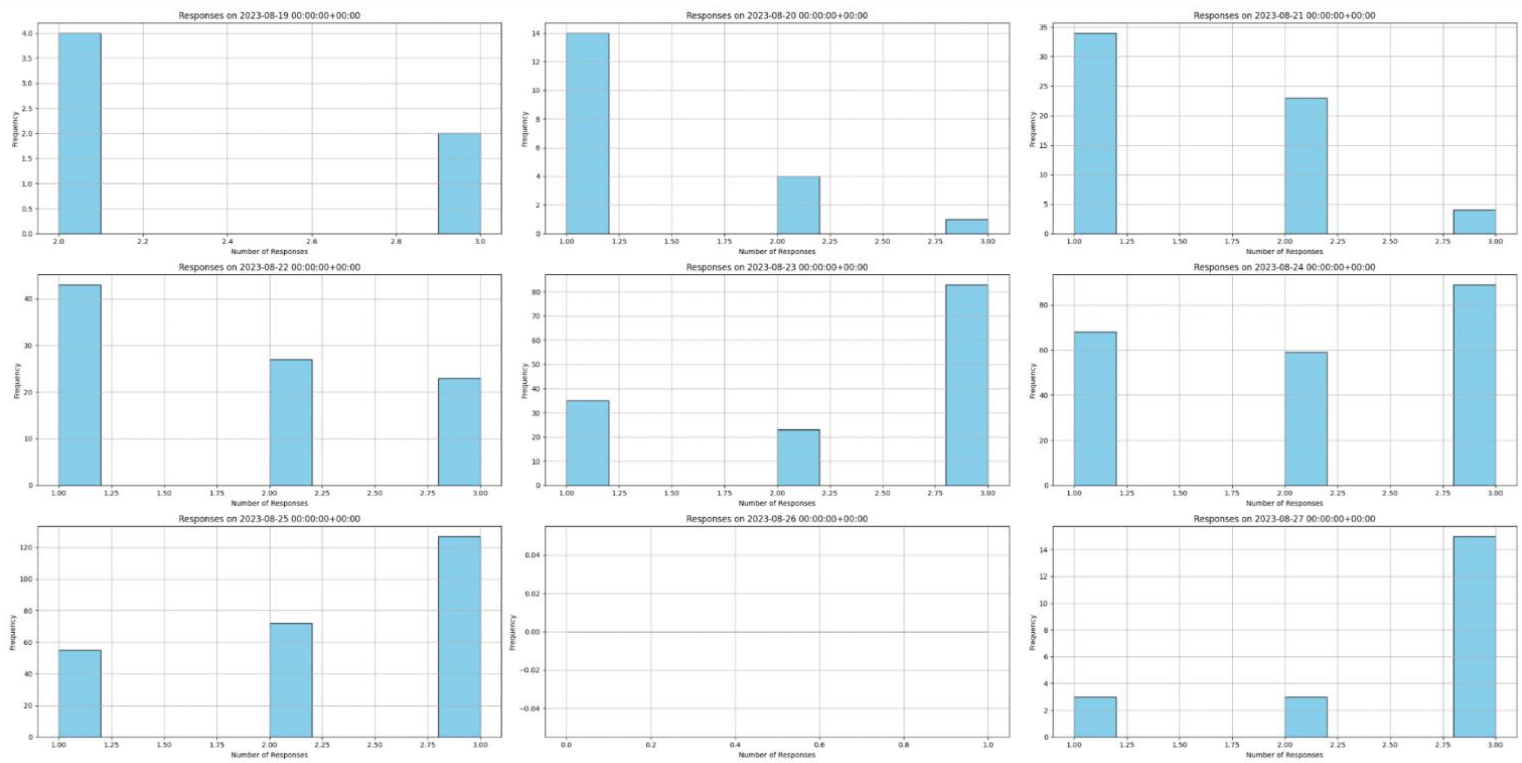








Data Visualization cont.



Data Visuals Analysis

- We start prompts on August 19th, which is a Saturday
- End Prompts on August 27th, which is a Sunday

```
day
2023-08-19    0.666667
2023-08-20    0.240506
2023-08-21    0.396104
2023-08-22    0.269697
2023-08-23    0.828402
2023-08-24    0.694444
2023-08-25    0.777419
2023-08-27    1.000000
```

```
 day
 2023-08-19      9
 2023-08-20     79
 2023-08-21    154
 2023-08-22    330
 2023-08-23    169
 2023-08-24    288
 2023-08-25    310
 2023-08-27     21
```

- This data is the proportion of successful compiles on each given day.
- It seems that towards the end of the week, from Wednesday the 23rd to Sunday the 27, compilations were very successful, even though there were a large amounts of prompts.

Data Visuals Analysis cont.

Question Id	day	
1	2023-08-19	0.000000
	2023-08-20	0.202899
	2023-08-21	0.280992
	2023-08-22	0.176991
	2023-08-23	0.596491
	2023-08-24	0.523364
	2023-08-25	0.550562
	2023-08-27	1.000000
2	2023-08-19	1.000000
	2023-08-20	1.000000
	2023-08-21	0.920000
	2023-08-22	0.651163
	2023-08-23	0.884615
	2023-08-24	0.716049
	2023-08-25	0.808989
	2023-08-27	1.000000
3	2023-08-19	1.000000
	2023-08-20	0.166667
	2023-08-21	0.500000
	2023-08-22	0.344262
	2023-08-23	0.965116
	2023-08-24	0.860000
	2023-08-25	0.909091
	2023-08-27	1.000000

- Here is the proportion of successful compiles of each question on each day
- Question 2 seems to be compiling successfully most of the time on any given day
- While Question 1 seems to struggle the most when it comes to compiling
- Question 1 is compiled the most overall, and it has the lowest success rate overall

Question Id	day	
1	2023-08-19	3
	2023-08-20	69
	2023-08-21	121
	2023-08-22	226
	2023-08-23	57
	2023-08-24	107
	2023-08-25	89
	2023-08-27	3
2	2023-08-19	4
	2023-08-20	4
	2023-08-21	25
	2023-08-22	43
	2023-08-23	26
	2023-08-24	81
	2023-08-25	89
	2023-08-27	3
3	2023-08-19	2
	2023-08-20	6
	2023-08-21	8
	2023-08-22	61
	2023-08-23	86
	2023-08-24	100
	2023-08-25	132
	2023-08-27	15

Feature Engineering

- I was interested in analyzing how time would affect the compile status of the data
- I engineered features that is consistent with that thought
- Since the model did not accept datetime format, using the Timestamp feature, I converted it to datetime and created the following features
    - Day of the Month
    - Day of the week
    - Weekend?
    - Hour of the Day
    - Minute of the day
- The creation of these features was to make the model more robust, and able to factor all aspects of the timestamps that were relevant
- Created the compile feature that is a binary label that consists of “compiled” or “not compiled”
    - Searched the stderr feature for terms: Traceback, NameError, SyntaxError, TypeError, KeyError

Models

- Feature Selection
  - Day of the week
  - Weekend?
  - Hour of the day
  - Minute of the day
  - Day of the Month
- Models:
  - Decision Tree Classifier
  - XGBoost Classifier
  - Random Forest Classifier
- Each model was implemented with a 5-Fold cross-validation, and its results were averaged

```
Decision Tree Classifier Results
Average Accuracy:  0.7985294117647059
Average Precision:  0.8567903358051515
Average Recall:  0.7893026992099095
Average F1-Score:  0.8213447106230347
XGBoost Classifier Results
Average Accuracy:  0.7691176470588236
Average Precision:  0.7993720053011322
Average Recall:  0.7969049075847321
Average F1-Score:  0.7973362472083338
Random Forest Classifier Results
Average Accuracy:  0.7830882352941176
Average Precision:  0.8393150013017203
Average Recall:  0.7781094390240965
Average F1-Score:  0.8070532746881426
```

Insights and Recommendations

- Overall, the models performed well, and I believed that there is something to be learned about the success and failures of gpt at a given time, and the amount of prompts being requested of gpt.
- The time that a prompt is requested can affect the efficiency and the performance of gpt, this is with little knowledge about how the prompts were constructed, but together with an analysis of the best way a to construct a prompt, we can boost the performance of gpt.